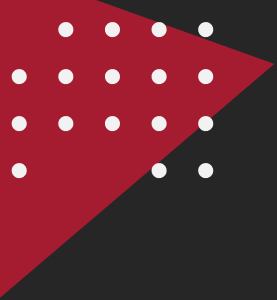




Caesar

Problem set



گفته می‌شود که سزار (Caesar)، برای رمزگذاری پیام‌های محرمانه (پنهان‌سازی به شکلی که قابل بازگشت باشد)، جایگاه هر حرف را جابه‌جا میکرد. مثلاً ممکن بود A را به‌جای B، B را به‌جای C، C را به‌جای D و الی آخر، و زمانی که به انتهای حروف الفبا میرسید دوباره به A بازمی‌گشت. پس سزار به‌جای HELLO گفتن به کسی، IFMMP را می‌نوشت. گیرندگان نامه‌های سزار باید آنها را با جابه‌جایی حروف در جهت عکس، رمزگشایی می‌کردند.

محرمانگی این سیستم رمزنگاری به سزار و گیرندگان پیام‌های او وابسته بود زیرا آنها از یک راز آگاه بودند: تعداد جایگاه‌هایی که او جابه‌جا کرده بود. این روش با توجه به استانداردهای امروزه، امن نیست، اما اگر اولین کسی باشید که چنین روشی را به کار می‌برد، به اندازه کافی امن بوده است!

به متن رمزگذاری نشده، متن ساده، به متن رمزگذاری شده، ciphertext و به آن راز، کلید می‌گویند.

این جدول نحوه رمزگذاری HELLO با کلید 1 را نشان میدهد که خروجی آن IFMMP میباشد:

plaintext	H	E	L	L	O
+key	1	1	1	1	1
=ciphertext	I	F	M	M	P

به بیان رسمی، این الگوریتم با حرکت دادن چرخشی هر حرف به اندازه k مکان در مجموعه حروف الفبای انگلیسی، پیام را رمزگذاری می‌کند. بیان رسمی‌تر این است که اگر p یک متن ساده باشد (رمزگذاری نشده)، p_i کاراکتر i ام در متن p و k یک کلید سری (یک عدد صحیح غیر منفی)، هر حرف c_i در متن رمزگذاری شده c ، به شکل زیر محاسبه می‌شود:

$$C_i = (p_i + k) \% 26$$

که در اینجا 26% به معنی باقیمانده‌ی تقسیم عددی بر 26 است. این فرمول ممکن است این رمزنگاری را از آن چیزی که هست سخت‌تر نشان دهد، اما این صرفاً یک راه فشرده برای توضیح این الگوریتم است. در این مسئله، A (یا a) را 0، B (یا b) را 1، ...، H (یا h) را 7 و Z (یا z) را 25 در نظر بگیرید. تصور کنید سزار می‌خواهد به کسی "Hi"، با کلید (k) 3، بگوید. پس متن ساده او "Hi" است که حرف اول آن، p_0 ، "H" یا همان 7 است؛ حرف دوم او، p_1 ، "i" یا همان 8 است. پس حرف اول پیام رمزگذاری شده او، c_0 ، "K" و حرف دوم او، c_1 ، "l" خواهد بود.

در فایلی به اسم `caesar.c` در فولدری به اسم `caesar`، برنامه‌ای بنویسید که در آن بتوان با رمزنگاری سزار یک پیام را رمزگذاری کرد. در حین اجرای برنامه، کاربر باید با یک آرگومان ورودی (`argument`)، تصمیم بگیرد پیامی که بعداً از او دریافت می‌شود با چه کلیدی رمزگذاری شود. نباید انتظار داشته باشیم که کلید کاربر حتماً یک عدد باشد. البته اگر عدد هم بود، باید یک عدد صحیح مثبت باشد.

برنامه‌ای طراحی کنید که با روش رمزنگاری سزار، یک پیام را رمزگذاری کند.

- برنامه خود را در فایل به اسم `caesar.c` در فولدری به اسم `caesar` پیاده‌سازی کنید.
- برنامه شما باید یک آرگومان ورودی دریافت کند که یک عدد صحیح نامنفی است؛ آن را `k` می‌نامیم.
- اگر برنامه شما بدون هیچ یا بیشتر از یک آرگومان دریافت کرد، باید یک پیام خطا (به انتخاب خودتان و با `printf`) چاپ کند و از `main` مقدار 1 را برگرداند که نشانه خطا است.
- اگر آرگومان وارد شده یک عدد نبود، برنامه شما باید پیام `Usage: ./caesar key` را چاپ کند و از `main`، 1 را برگرداند.
- انتظار نداشته باشید `k` کوچک‌تر یا مساوی 26 باشد، برنامه شما باید برای تمام اعداد صحیح نامنفی از 26 تا 2^{31} به درستی کار کند. اگر عدد کاربر به قدری بزرگ بود که در یک `int` جا نمی‌شد، نگران خراب شدن برنامه‌تان نباشید زیرا باید به یاد داشته باشید که `int` می‌تواند سرریز شود. حتی اگر `k` بزرگتر از 26 بود، حروف پیام ورودی در خروجی، باید حروف الفبا بمانند. مثلاً اگر `k`، 27 بود، اگرچه “\”، 27 امین جایگاه بعد از “A” است، “A” نباید تبدیل به “\” شود (ASCII). طبق این جدول، به شرطی که از “Z” به “A” برگردید، “B”، 27 امین جایگاه بعد از “A” است بنابراین “A” باید تبدیل به “B” شود.

- برنامه شما باید برای دریافت ورودی `plaintext:` را با 2 فاصله و بدون خط جدید چاپ کند و از کاربر با استفاده از `get_string` یک رشته دریافت کند.
- برنامه شما باید برای نشان دادن خروجی، `ciphertext:` را با 1 فاصله و بدون خط جدید و در ادامه آن، پیام رمزگذاری شده بر اساس کلید `k` را چاپ کند. کاراکترهای غیر از الفبا باید بدون تغییر در جای خود چاپ شوند.
- برنامه شما باید حروف کوچک و بزرگ را حفظ کند: حروف بزرگ در پیام رمزگذاری شده باید بزرگ بمانند و حروف کوچک نیز باید کوچک بمانند.
- بعد از چاپ خروجی، برنامه باید یک خط جدید چاپ کرده و با 0 از `main` خارج شود.

بیایید این مسئله را قدم به قدم پیش ببریم.

شبه کد

اول یک تابع `main` در `caesar.c` بنویسید که این برنامه را با شبه کد پیاده‌سازی می‌کند.

راهنمایی: بیشتر از یک راه برای انجام این کار وجود دارد و یکی از آنها این است: می‌توانید شبه کدهای ما را برای خودتان ویرایش کنید اما آن را کپی نکنید!

```
int main(int argc, string argv[]) {  
    // Make sure program was run with just one command-line argument  
  
    // Make sure every character in argv[1] is a digit  
  
    // Convert argv[1] from a `string` to an `int`  
  
    // Prompt user for plaintext  
  
    // For each character in the plaintext:  
  
        // Rotate the character if it's a letter  
}
```

بیا ببینیم اول کدی بنویسیم که اجرا شدن برنامه با یک آرگومان را بررسی کند و سپس قابلیت‌های بیشتر را اضافه می‌کنیم. مخصوصاً `main` را طوری طراحی کنید که اگر کاربر آرگومانی وارد نکرد یا تعداد آرگومان‌های داده شده بیشتر از 1 بود، `"Usage: ./caesar key\n"` را چاپ کند و با بازگرداندن 1، از برنامه خارج شود. اگر کاربر دقیقاً یک آرگومان را وارد کرد، برنامه باید به سادگی بدون انجام کاری، 0 را برگرداند. برنامه شما باید اینگونه عمل کند:

```
$ ./caesar Usage:
./caesar key

$ ./caesar 1 2 3
Usage: ./caesar key

$ ./caesar 1
```

راهنمایی:

- به یاد داشته باشید که می‌توانید با `printf` چاپ کنید.
- به یاد داشته باشید که یک تابع می‌تواند با `return` یک مقدار را برگرداند.
- به یاد داشته باشید که `argc` حاوی تعداد آرگومان‌های داده شده به یک برنامه به علاوه اسم برنامه است.

حال که برنامه شما ورودی را همانطور که باید قبول می‌کند، وقت آن است که قدم بعدی را بردارید.

در `caesar.c`، زیر `main`، یک تابع به اسم `only_digits` بنویسید که یک رشته را به عنوان آرگومان دریافت می‌کند. اگر این رشته فقط حاوی اعداد بین 0 تا 9 بود، “true”، و در غیر این صورت “false” را برمی‌گرداند. مطمئن شوید که `prototype` این تابع را در بالای `main` نیز اضافه کنید.

```
bool only_digits(string s);
```

راهنمایی:

- احتمالاً به یک `prototype` مثل این نیاز دارید:
در بالای فایل خود `cs50.h` را `include` کنید تا کامپایلر `string` و `bool` را شناسایی کند.
- به یاد داشته باشید که یک `string` در واقع یک آرایه از `char` ها می‌باشد.
- به یاد داشته باشید که `strlen` که در `string.h` تعریف شده، طول یک رشته را محاسبه می‌کند.
- شاید `isdigit` که در `ctype.h` تعریف شده، طبق <https://manual.cs50.io> کمک کننده باشد؛ اما توجه داشته باشید که یکی یکی کاراکترها را بررسی می‌کند.

سپس `main` را طوری تغییر دهید که `only_digits` را روی `argv[1]` فراخوانی کند. اگر آن تابع `false` را

برگرداند، تابع `main` باید `"Usage: ./caesar key\n"` را چاپ کند و 1 را برگرداند. در غیر این صورت

`main` باید 0 را برگرداند. برنامه شما باید مانند زیر عمل کند:

```
$ ./caesar  
42 $  
./caesar  
banana  
  
Usage:  
./caesar key
```

استفاده از کلید

حال `main` را طوری تغییر دهید که `arg[1]` را به یک عدد صحیح تبدیل کند. احتمالاً `atoi`، که در `stdlib.h` تعریف

شده، کمک کننده باشد (<https://manual.cs50.io>). سپس از `get_string` برای گرفتن متن ساده با `"plaintext: "` استفاده کنید.

بعد تابعی به اسم مثلاً `rotate` پیاده‌سازی کنید که به عنوان ورودی یک `char` و یک `int` را گرفته، در صورتی که کاراکتر ورودی، حرف الفبایی بود، آن را به اندازه‌ی مشخص شده جابه‌جا کند و در صورت لزوم از `"z"` به `"a"` (`"Z"` به `"A"`) بازگردد. اگر کاراکتر ورودی، حرف الفبایی نبود، تابع باید همان کاراکتر را بدون تغییر برگرداند.

```
char rotate(char c, int n);
```

- احتمالاً به یک prototype مثل این نیاز خواهید داشت:

```
rotate('A', 1)
```

یک فراخوانی تابع مانند:

```
rotate('A', 27)
```

یا حتی اینگونه؛ که باید "B" را برگرداند:

```
rotate('!', 13)
```

و یک فراخوانی تابع مانند این، که باید "!" را برگرداند:

- به یاد داشته باشید که میتوانید یک `char` را به طور صریح با `(int)` به یک عدد صحیح و یک عدد صحیح را با `(char)` به یک کاراکتر تبدیل کنید. یا میتوانید این کار را به طور ضمنی با رفتار کردن با هر کدام به عنوان دیگری انجام دهید.

- ممکن است بخواهید، همانطور که در محاسبات ریاضی انجام می‌دهید، مقدار ASCII حرف "A" را از حروف بزرگ کسر کنید تا "A" را به عنوان 0، "B" را به عنوان 1 و به همین ترتیب، ادامه دهید و پس از پایان محاسبات، همان مقدار را دوباره اضافه کنید تا به شکل اولیه برگردد.
 - همانند مورد قبل، احتمالاً می‌خواهید مقدار ASCII حرف "a" را از حروف کوچک کسر کنید تا "a" را به عنوان 0، "b" را به عنوان 1 و به همین ترتیب، ادامه دهید. پس از پایان محاسبات، همان مقدار را دوباره اضافه کنید.
 - شاید بعضی دیگر از توابعی که در `ctype.h` تعریف شده‌اند مفید باشند: [/https://manual.cs50.io](https://manual.cs50.io)
 - احتمالاً % را هنگام "بازگشت به ابتدا" از مقداری مانند 25 به 0 مفید بیابید.
- سپس تابع `main` را به گونه‌ای تغییر دهید که `"ciphertext: "` را چاپ کند و سپس روی هر کاراکتر از متن ساده حرکت کرده و با فراخوانی تابع `rotate`، نتیجه نهایی را چاپ کند.

راهنمایی:

- به یاد داشته باشید که `printf` می‌تواند یک کاراکتر را با `%c` چاپ کند.
- اگر هنگام فراخوانی `printf` خروجی‌ای دریافت نمی‌کنید، احتمالاً بخاطر این است که در حال چاپ کردن کاراکترهایی خارج از محدوده ASCII که از 0 تا 127 است هستید. برای بررسی اینکه چه مقادیری را چاپ می‌کنید، موقتاً کاراکترها را به صورت اعداد (با استفاده از `%i` و `%c`) چاپ کنید!

درستی

```
check50 cs50/problems/2025/x/caesar
```

استایل

```
style50 caesar.c
```

بعد از کامپایل کردن کد خود با `make`، مثل زیر عمل کنید:

```
debug50 ./caesar KEY
```

که در اینجا `key` همان کلیدی است که به عنوان آرگومان وارد می‌کنید. توجه داشته باشید که اجرای دستور روبه‌رو باعث می‌شود که برنامه شما با درخواست یک کلید (`key`) از کاربر پایان یابد.

با اجرا کردن صورت زیر در Terminal پاسخ خود را ارسال کنید.

```
submit50 cs50/problems/2025/x/caesar
```

CS50x Iran

Harvard's Computer Science 50x Iran

